

1 Vorspann: Sprachen

Def. 1.1: Ein *Alphabet* ist eine nicht leere, endliche Menge von *Zeichen*.

Vorlesung:
21.04.26

Zeichen sind hier beliebige abstrakte Symbole.

Bsp.: für Alphabete, die in dieser Vorlesung, im täglichen Umgang mit Computern oder in der Forschung an unserem Lehrstuhl eine Rolle spielen

- $\{a, \dots, z\}$
- $\{0, 1\}$
- $\{\text{rot}, \text{gelb}, \text{grün}\}$ (Ampelfarben)
- Die Menge aller ASCII-Symbole
- Die Menge aller Statements eines Computerprogramms

Wir verwenden typischerweise den griechischen Buchstaben Σ als Namen für ein Alphabet und die lateinischen Buchstaben a, b, c als Namen für Zeichen. Im Folgenden sei Σ immer ein beliebiges Alphabet.

Def. 1.2: Ein *Wort* ist eine endliche Folge von Elementen aus Σ . Die leere Folge nennen wir das *leere Wort* geschrieben als ε . Wir bezeichnen die Menge aller Wörter mit Σ^* und die Menge aller nicht leeren Wörter mit $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$.

Hier die induktive Definition.

1. $\varepsilon \in \Sigma^*$.
2. $a \in \Sigma$ und $w \in \Sigma^*$ impliziert $a.w \in \Sigma^*$.

◇

Hier einige Konventionen:

- Wir schreiben ein Wort immer ohne Trennsymbole wie z.B. Komma. Also **einhorn** statt **e,i,n,h,o,r,n**.
- Wir verwenden ε für das leere Wort.
- Wir verwenden typischerweise u, v, w als Namen für Wörter.

Bsp.: für Wörter über $\Sigma = \{a, \dots, z\}$

- rambo (Länge 5)
- eis, ies (beide Länge 3 aber ungleich)
- ε (Länge 0)

Aus der induktiven Definition ergibt sich das folgende Beweisprinzip für Aussagen $\varphi(w)$ über Worte w :

$$\frac{\varphi(\varepsilon) \quad \forall a, w. \varphi(w) \Rightarrow \varphi(a.w)}{\forall w \in \Sigma^*. \varphi(w)}$$

Dieses Prinzip können wir auch verwenden um Funktionen auf Worten zu definieren. Als erstes Beispiel definieren wir die Länge eines Wortes, d.h. die Anzahl der Symbole des Wortes.

Def. 1.3 (Länge eines Worts): Die *Länge* eines Worts, $|_| : \Sigma^* \rightarrow \mathbb{N}$, wird für alle $w \in \Sigma^*$ induktiv definiert durch

1. $|\varepsilon| = 0$;
2. für alle $a \in \Sigma$ und $w \in \Sigma^*$: $|a.w| = 1 + |w|$.

◇

Wörter lassen sich „verketten“/„aneinanderhängen“. Die entsprechende Operation heißt *Konkatenation*, geschrieben „ $\cdot$ “ (wie Multiplikation).

Def. 1.4 (Konkatenation von Wörtern): Die *Konkatenation*, $\cdot : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, wird für alle $v \in \Sigma^*$ induktiv definiert durch

1. $\varepsilon \cdot v = v$;
2. für alle $a \in \Sigma$ und $u \in \Sigma^*$: $(a.u) \cdot v = a.(u \cdot v)$.

◇

Lemma 1.1: Eigenschaften der Konkatenation: $(\Sigma^*, \varepsilon, \cdot)$ ist ein Monoid:

1. $\varepsilon \cdot v = v$ (leeres Wort ist linksneutrales Element);
2. $v \cdot \varepsilon = v$ (leeres Wort ist rechtsneutrales Element);
3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ (Assoziativität).

BEWEIS: 1. $\varepsilon \cdot v = v$ (Definition)

2. $v \cdot \varepsilon = v$ Induktion über v :

- IA: $(v = \varepsilon) \ \varepsilon \cdot \varepsilon = \varepsilon$ (Definition)
- IS: $(v \rightsquigarrow a.v) \ (a.v) \cdot \varepsilon = a.(v \cdot \varepsilon) \stackrel{IV}{=} a.v$

3. $u \cdot (v \cdot w) = (u \cdot v) \cdot w$ Induktion über u :

- IA: $(u = \varepsilon) \ \varepsilon \cdot (v \cdot w) = (v \cdot w) = (\varepsilon \cdot v) \cdot w$ (Definition)
- IS: $(u \rightsquigarrow a.u) \ (a.u) \cdot (v \cdot w) \stackrel{Def}{=} a.(u \cdot (v \cdot w)) \stackrel{IV}{=} a.((u \cdot v) \cdot w) \stackrel{Def}{=} ((a.u) \cdot v) \cdot w$

□

Bsp.:

- $\mathbf{eis} \cdot \mathbf{rambo} = \mathbf{eisrambo}$
- $\mathbf{rambo} \cdot \varepsilon = \mathbf{rambo} = \varepsilon \cdot \mathbf{rambo}$

Der Konkatenationsoperator „ $\cdot$ “ wird oft weggelassen (ähnlich wie der Multiplikationsoperator in der Arithmetik). Ebenso können durch die Assoziativität Klammern weggelassen werden.

$w_1 w_2 w_3$ steht also auch für $w_1 \cdot w_2 \cdot w_3$, für $(w_1 \cdot w_2) \cdot w_3$ und für $w_1 \cdot (w_2 \cdot w_3)$

Bemerkung: Die Zeichenfolge $\mathbf{rambo}\varepsilon$ ist *kein* Wort. Diese Zeichenfolge ist lediglich eine Notation für eine Konkatenationsoperation, die ein Wort der Länge 5 (nämlich $\mathbf{rambo}$) beschreibt.

Wörter lassen sich außerdem *potenzieren*:

Def. 1.5: Die *Potenzierung* von Wörtern, $\cdot^{\cdot} : \Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$, ist induktiv definiert durch

1. $w^0 = \varepsilon$
2. $w^{n+1} = w \cdot w^n$ ◇

Bsp.: $\mathbf{eis}^3 \stackrel{(2.)}{=} \mathbf{eis} \cdot \mathbf{eis}^2 \stackrel{\text{zweimal } (2.)}{=} \mathbf{eis} \cdot \mathbf{eis} \cdot \mathbf{eis} \cdot \mathbf{eis}^0 \stackrel{(1.)}{=} \mathbf{eis} \cdot \mathbf{eis} \cdot \mathbf{eis} \cdot \varepsilon = \mathbf{eiseiseis}$

Def. 1.6: Eine *Sprache* über Σ ist eine Menge $L \subseteq \Sigma^*$. Gleichwertig: $L \in \mathcal{P}(\Sigma^*)$. ◇

Bsp.:

- $\{\text{eis}, \text{rambo}\}$
- $\{w \in \{0, 1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$
- $\{\}$ (die „leere Sprache“)
- $\{\varepsilon\}$ (ist verschieden von der leeren Sprache)
- Σ^*

Sämtliche Mengenoperationen sind auch Sprachoperationen, insbesondere Schnitt ($L_1 \cap L_2$), Vereinigung ($L_1 \cup L_2$), Differenz ($L_1 \setminus L_2$) und Komplement ($\overline{L_1} = \Sigma^* \setminus L_1$).

Weitere Operationen auf Sprachen sind Konkatenation und Potenzierung, sowie der *Kleene-Abschluss*.

Def. 1.7 (Konkatenation und Potenzierung von Sprachen): Seien $U, V \subseteq \Sigma^*$. Dann ist die *Konkatenation* von U und V definiert durch

$$U \cdot V = \{uv \mid u \in U, v \in V\}$$

und die *Potenzierung* von U induktiv definiert durch

1. $U^0 = \{\varepsilon\}$
2. $U^{n+1} = U \cdot U^n$

◇

Bsp.:

- $\{\text{eis}, \varepsilon\} \cdot \{\text{rambo}\} = \{\text{eisrambo}, \text{rambo}\}$
- $\{\text{eis}, \varepsilon\} \cdot \{\} = \{\}$
- $\{\}^0 = \{\varepsilon\}$
- $\{\}^4 = \{\}$
- $\{\text{eis}, \varepsilon\}^2 = \{\varepsilon, \text{eis}, \text{eiseis}\}$

Wie bei der Konkatenation von Wörtern dürfen wir den Konkatenationsoperator auch weglassen.

Def. 1.8 (Kleene-Abschluss, Kleene-Stern, Kleene-Plus): Gegeben eine Sprache $U \subseteq \Sigma^*$, definieren wir den *Kleene-Abschluss* U^* wie folgt.

$$U^* = \bigcup_{n \in \mathbb{N}} U^n$$

Wie nennen den Operator $\cdot^* : \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$, der für eine gegebene Sprache den Kleene-Abschluss liefert, den *Kleene-Stern*. Analog definieren wir den *Kleene-Plus* Operator $\cdot^+ : \mathcal{P}(\Sigma^*) \rightarrow \mathcal{P}(\Sigma^*)$ als

$$U^+ = \bigcup_{n \geq 1} U^n.$$

◇

Es gilt also $U^* = U^+ \cup \{\varepsilon\}$.

Zum Abschluss eine alternative Definition, die in manchen Textbüchern oder manchmal in der Literatur verwendet wird. In diesem Skript werden wir sie nicht weiter verwenden.

Def. 1.9 (Alternative Definition für Σ^*): Sei $n \in \mathbb{N}$.

- $\Sigma^n = \{1, \dots, n\} \rightarrow \Sigma$ Worte der Länge n
- insbesondere $\Sigma^0 = \{\}$ $\rightarrow \Sigma$ enthält nur ein Element, ε
- $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$
- Konkatination: $u \in \Sigma^n, v \in \Sigma^m \Rightarrow u \cdot v \in \Sigma^{n+m}$ definiert durch

$$(u \cdot v)(i) = \begin{cases} u(i) & 1 \leq i \leq n \\ v(i - n) & n < i \leq n + m \end{cases}$$

◇

2 Reguläre Sprachen und endliche Automaten

Vorlesung:
22.04.26

Wie können wir potentiell unendlich große Mengen von Wörtern darstellen? Eine Lösung für dieses Problem sahen wir bereits im vorherigen Kapitel, als wir die (unendlich große) Menge der binär codierten Primzahlen mit Hilfe der folgenden Sprache darstellten.

$$L_{\text{prim}} = \{w \in \{0,1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$$

Ein weiteres Beispiel ist die folgende Sprache.

$$L_{\text{even}} = \{w \in \{0,1\}^* \mid \text{die Anzahl der Einsen in } w \text{ ist gerade.}\}$$

Eine häufig interessante Fragestellung für ein gegebenes Wort w und eine Sprache L ist: „Ist w in L enthalten?“ (Also: „Gilt $w \in L$?“) Wir nennen dieses Entscheidungsproblem das *Wortproblem*. Eine konkrete Instanz des Wortproblems wäre z.B. „1100101 $\in L_{\text{prim}}$?“ oder „1100101 $\in L_{\text{even}}$?“

Die obige Darstellung der unendlichen Mengen L_{prim} und L_{even} ist zwar sehr kompakt, wir können daraus aber nicht direkt ein Vorgehen zur Lösung des Wortproblems ableiten. Wir müssen zunächst verstehen, was die Begriffe „Binärcodierung“, „Primzahl“ oder „gerade Anzahl“ bedeuten und für L_{prim} und L_{even} jeweils einen Algorithmus zur Entscheidung entwickeln.

In diesem Kapitel werden wir mit *endlichen Automaten* einen weiteren Formalismus kennenlernen, um (potentiell unendlich große) Mengen von Wörtern darzustellen. Ein Vorteil dieser Darstellung ist, dass es einen einheitlichen und effizienten Algorithmus für das Wortproblem gibt. Wir werden aber auch sehen, dass sich nicht jede Sprache (z.B. L_{prim}) mit Hilfe eines endlichen Automaten darstellen lässt.

2.1 Endliche Automaten

Wir beschreiben zunächst informell die Bestandteile eines endlichen Automaten:

Endliches Band (read-only; jede Zelle enthält ein $a_i \in \Sigma$; der Inhalt des Bands ist das *Eingabewort* bzw. die *Eingabe*)

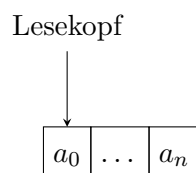


Abb. 1: Endliches Band

Lesekopf

- Der *Lesekopf* zeigt auf ein Feld des Bands, oder hinter das letzte Feld.
- Er bewegt sich feldweise nach rechts; andere Bewegungen (Vor- bzw. Zurückspulen) sind nicht möglich.
- Wenn er hinter das letzte Zeichen zeigt, *stoppt* der Automat. Er muss sich nun „entscheiden“ ob er das Wort *akzeptiert* oder nicht.

Zustände Zu jedem Zeitpunkt befindet sich der Automat in einem Zustand q aus einer *endlichen* Zustandsmenge Q .

Startzustand $q^{\text{init}} \in Q$

Akzeptierende Zustände $F \subseteq Q$

Transitionsfunktion Im Zustand q beim Lesen von a gehe in Zustand $q' := \delta(q, a)$.

Der endliche Automat akzeptiert eine Eingabe, falls er in einem akzeptierenden Zustand stoppt.

Bsp. 2.1: Aufgabe:

„Erkenne alle Stapel von Macarons, in denen höchstens ein grünes Macaron vorkommt.“



Ein passendes Alphabet wäre $\Sigma = \{\text{grün}, \text{nicht-grün}\}$. Wir definieren die folgenden Zustände. (Die Metapher hier ist: „wenn ich mehr als ein grünes Macaron esse, wird mir übel, und das wäre nicht akzeptabel“.)

Zustand	Bedeutung
q_0	„alles gut“
q_1	„mir wird schon flau“
q_2	„mir ist übel“

Der Startzustand ist q_0 . Akzeptierende Zustände sind q_0 und q_1 . Die Transitionsfunktion δ ist durch die folgende Tabelle gegeben.

¹Von links nach rechts:

By Mariajudit - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=48726001>
 By Michelle Naherny - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=44361114>
 By Keven Law - originally posted to Flickr as What's your Colour???, CC BY-SA 2.0, <https://commons.wikimedia.org/w/index.php?curid=6851868>

	grün	nicht-grün	
q_0	q_1	q_0	wechsle nach q_1 falls grün , ansonsten verweile
q_1	q_2	q_1	wechsle nach q_2 falls grün , ansonsten verweile
q_2	q_2	q_2	verweile, da es nichts mehr zu retten gibt

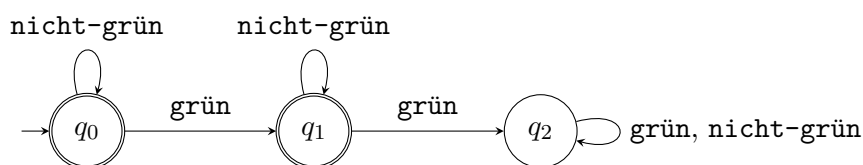
Def. 2.1 (DEA): Ein *deterministischer endlicher Automat* (DEA)² ist ein 5-Tupel

$$\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F).$$

Dabei ist

- Σ ein Alphabet,
- Q eine *endliche* Menge, deren Elemente wir *Zustände* nennen,
- $\delta : Q \times \Sigma \rightarrow Q$ eine Funktion, die wir *Transitionsfunktion* nennen,
- $q^{\text{init}} \in Q$ ein Zustand, den wir *Startzustand* nennen und
- $F \subseteq Q$ eine Teilmenge der Zustände, deren Elemente wir *akzeptierende Zustände* nennen. $\diamond$

DEAs lassen sich auch graphisch darstellen. Dabei gibt man für den Automaten einen gerichteten Graphen an. Die Knoten des Graphen sind die Zustände, und mit Zeichen beschriftete Kanten zeigen, welchen Zustandsübergang die Transitionsfunktion für das nächste Zeichen erlaubt. Der Startzustand ist mit einem unbeschrifteten Pfeil markiert, akzeptierende Zustände sind doppelt eingekreist. Hier ist die graphische Darstellung von A_{Macaron} aus Beispiel 2.1:



Die folgenden beiden Definitionen erlauben uns mit Hilfe eines DEA eine Sprache zu charakterisieren. Hierbei sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ein DEA.

Def. 2.2 (Induktive Erweiterung von δ auf Wörter): Die *induktive Erweiterung* von $\delta : Q \times \Sigma \rightarrow Q$ auf Wörter, $\tilde{\delta} : Q \times \Sigma^* \rightarrow Q$, ist (induktiv) definiert durch

1. $\tilde{\delta}(q, \varepsilon) = q$ (Wortende erreicht)

²engl. DFA $\triangleq$ deterministic finite automaton

$$2. \tilde{\delta}(q, aw) = \tilde{\delta}(\delta(q, a), w) \quad (\text{Rest im Folgezustand verarbeiten}) \quad \diamond$$

Def. 2.3 (Durch einen DEA akzeptierte Sprache): Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$. Ein Wort $w \in \Sigma^*$ wird von $\mathcal{A}$ *akzeptiert*, falls $\tilde{\delta}(q^{\text{init}}, w) \in F$. Die *von $\mathcal{A}$ akzeptierte Sprache*, geschrieben $L(\mathcal{A})$, ist die Menge aller Wörter, die von $\mathcal{A}$ akzeptiert werden. D.h.,

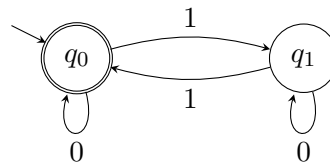
$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \tilde{\delta}(q^{\text{init}}, w) \in F\}.$$

Eine durch einen DEA akzeptierte Sprache heißt *regulär*. $\diamond$

Bsp. 2.2: Ein möglicher DEA für die Sprache

$$L_{\text{even}} = \{w \in \{0, 1\}^* \mid \text{Die Anzahl der Einsen in } w \text{ ist gerade.}\}$$

aus der Einleitung dieses Kapitels hat die folgende graphische Repräsentation.



Frage: Angenommen wir haben zwei DEAs A_1 und A_2 , können wir dann immer einen DEA konstruieren der den Durchschnitt $L(A_1) \cap L(A_2)$ akzeptiert?

Idee: Lasse beide DEAs parallel laufen, akzeptiere nur wenn beide DEAs akzeptieren.

Frage: Lassen sich zwei *parallel laufende* DEAs in einem einzigen DEA implementieren?

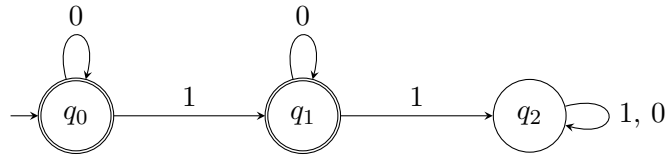
Die nächste Konstruktion und der drauf folgende Satz liefern eine positive Antwort auf diese Frage.

Def. 2.4 (Produktautomat für Durchschnitt): Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\text{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\text{init}}, F_2)$ DEAs. Wir definieren den *Produktautomaten für Durchschnitt* als den DEA $A_\cap = (\Sigma, Q_\cap, \delta_\cap, q_\cap^{\text{init}}, F_\cap)$ mit den folgenden Komponenten.

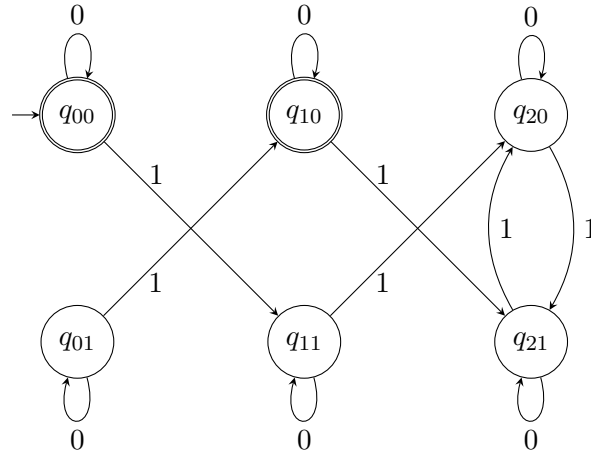
$$\begin{aligned} Q_\cap &= Q_1 \times Q_2 \\ \delta_\cap((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, a)) \quad , \quad \text{für alle } a \in \Sigma \\ q_\cap^{\text{init}} &= (q_1^{\text{init}}, q_2^{\text{init}}) \\ F_\cap &= F_1 \times F_2 \end{aligned} \quad \diamond$$

Im folgenden Beispiel sei A_1 der DEA über dem Alphabet $\Sigma = \{0, 1\}$, dessen graphische Repräsentation nahezu mit A_{Macaron} identisch ist.

Vorlesung:
28.04.26



Bsp. 2.3: Der Produktautomat für den Durchschnitt von A_1 und A_{even} hat die folgende graphische Repräsentation, wobei wir um Platz zu sparen „ q_{ij} “ statt „ (q_i, q_j) “ schreiben.



Satz 2.1: Für beliebige DEAs A_1 und A_2 akzeptiert der entsprechende Produktautomat für Durchschnitt die Sprache $L(A_1) \cap L(A_2)$.

BEWEIS: ³ Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\text{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\text{init}}, F_2)$ DEAs und $A_\cap = (\Sigma, Q_\cap, \delta_\cap, q_\cap^{\text{init}}, F_\cap)$ der zugehörige Produktautomat für den Durchschnitt. Wir zeigen zunächst per Induktion über w , dass für alle $w \in \Sigma^*$, für alle $q_1 \in Q_1$ und für alle $q_2 \in Q_2$ die folgende Gleichung $\varphi(w)$ gilt.

$$\varphi(w) := \tilde{\delta}_\cap((q_1, q_2), w) = (\tilde{\delta}_1(q_1, w), \tilde{\delta}_2(q_2, w))$$

Der Induktionsanfang $\varphi(\varepsilon)$ folgt dabei direkt aus Def. 2.2.

$$\tilde{\delta}_\cap((q_1, q_2), \varepsilon) = (q_1, q_2)$$

³Dieser erste Beweis ist außergewöhnlich detailliert. In den folgenden Beweisen werden wir einfache Umformungen zusammenfassen.

Den Induktionsschritt $\varphi(w) \rightsquigarrow \varphi(a.w)$ zeigen wir mit Hilfe der folgenden Umformungen, wobei $a \in \Sigma$ ein beliebiges Zeichen und $w \in \Sigma^*$ ein beliebiges Wort.

$$\begin{aligned}
 \tilde{\delta}_\cap((q_1, q_2), aw) &\stackrel{\text{Def. 2.2}}{=} \tilde{\delta}_\cap(\delta_\cap((q_1, q_2), a), w) \\
 &\stackrel{\text{Def. } \delta_\cap}{=} \tilde{\delta}_\cap((\delta_1(q_1, a), \delta_2(q_2, a)), w) \\
 &\stackrel{\text{I.V.}}{=} (\tilde{\delta}_1(\delta_1(q_1, a), w), \tilde{\delta}_2(\delta_2(q_2, a), w)) \\
 &\stackrel{\text{Def. 2.2}}{=} (\tilde{\delta}_1(q_1, aw), \tilde{\delta}_2(q_2, aw))
 \end{aligned}$$

Schließlich zeigen wir $L(A_\cap) = L(A_1) \cap L(A_2)$ mit Hilfe der folgenden Umformungen für ein beliebiges $w \in \Sigma^*$.

$$\begin{aligned}
 w \in L(A_\cap) &\stackrel{\text{Def. 2.3}}{\text{gdw}} \tilde{\delta}_\cap(q_\cap^{\text{init}}, w) \in F_\cap \\
 &\stackrel{\text{Def. } q_\cap^{\text{init}}}{\text{gdw}} \tilde{\delta}_\cap((q_1^{\text{init}}, q_2^{\text{init}}), w) \in F_\cap \\
 &\stackrel{\text{gdw}}{=} (\tilde{\delta}_1(q_1^{\text{init}}, w), \tilde{\delta}_2(q_2^{\text{init}}, w)) \in F_\cap \\
 &\stackrel{\text{Def. } F_\cap}{\text{gdw}} \tilde{\delta}_1(q_1^{\text{init}}, w) \in F_1 \text{ und } \tilde{\delta}_2(q_2^{\text{init}}, w) \in F_2 \\
 &\stackrel{\text{Def. 2.3}}{\text{gdw}} w \in L(A_1) \text{ und } w \in L(A_2)
 \end{aligned}$$

□

2.2 Minimierung endlicher Automaten

Beobachtung: Der Zustand q_{01} im Beispiel 2.3 scheint nutzlos. Wir charakterisieren die Nützlichkeit eines Zustands in einem DEA $\mathcal{A} = (Q, \Sigma, \delta, q^{\text{init}}, F)$ formal wie folgt.

Def. 2.5: Ein Zustand $q \in Q$ heißt *erreichbar*, falls ein $w \in \Sigma^*$ existiert, sodass $\tilde{\delta}(q^{\text{init}}, w) = q$. ◇

Bemerkung: Die Menge der erreichbaren Zustände kann mit dem folgenden Verfahren in $O(|Q| \cdot |\Sigma|)$ berechnet werden.

- Fasse $\mathcal{A}$ als Graph auf.
- Wende Tiefensuche ab q^{init} an und markiere dabei alle besuchten Zustände.
- Genau die markierten Zustände sind erreichbar.

Beobachtung: Auch nach dem Entfernen der nicht erreichbaren Zustände q_{01} und q_{10} scheint der DEA aus Bsp. 2.3 unnötig groß zu sein. Das Verhalten des DEA in den Zuständen q_{11} , q_{20} und q_{21} ist sehr ähnlich. Wir charakterisieren diese „Ähnlichkeit“ formal wie folgt.

Def. 2.6: Zwei Zustände $p, q \in Q$ eines DEA sind *äquivalent*, geschrieben $p \equiv q$, falls

$$\forall w \in \Sigma^* : \tilde{\delta}(p, w) \in F \text{ gdw } \tilde{\delta}(q, w) \in F \quad \diamond$$

Bsp. 2.4: Für Bsp. 2.3 gilt: Die Zustände q_{11} , q_{20} und q_{21} sind paarweise äquivalent. Die Zustände q_{00} und q_{10} sind äquivalent. Keine weiteren Zustandspaare sind äquivalent.

Geschrieben als Menge von Paaren sieht die Relation $\equiv \subseteq Q \times Q$ also wie folgt aus:

$$\{(q_{00}, q_{10}), (q_{10}, q_{00}), (q_{11}, q_{20}), (q_{20}, q_{11}), (q_{20}, q_{21}), (q_{21}, q_{20}), (q_{21}, q_{11}), (q_{11}, q_{21})\}$$

Lemma 2.2: Die Relation „ $\equiv$ “ ist eine *Äquivalenzrelation*.

BEWEIS: Zu zeigen ist, dass $\equiv$ reflexiv, transitiv und symmetrisch ist.

- Die Relation $\equiv$ ist offensichtlich reflexiv.
- Die Symmetrie und Transitivität von $\equiv$ folgen aus der Symmetrie und Transitivität der logischen Interpretation von „genau dann, wenn“ (gdw).

□

Bsp. 2.5: Für den DEA aus Bsp. 2.3 hat die Relation $\equiv$ drei Äquivalenzklassen.⁴

$$\begin{aligned} [q_{00}] &= \{q_{00}, q_{10}\}, \\ [q_{01}] &= \{q_{01}\}, \\ [q_{11}] &= \{q_{11}, q_{20}, q_{21}\} \end{aligned}$$

Idee: „Verschmelze“ alle Zustände aus einer Äquivalenzklasse zu einem einzigen Zustand. Bedenken: Bei einem DEA hat jeder Zustand für jedes Zeichen einen Nachfolger. Wenn wir Zustände verschmelzen, könnte es mehrere Nachfolger geben und das Resultat wäre kein wohldefinierter DEA mehr.

Das folgende Lemma zeigt, dass unsere Bedenken nicht gerechtfertigt sind. Sind zwei Zustände äquivalent, so sind auch für jedes Zeichen ihre Nachfolger äquivalent.

Lemma 2.3: Für alle $p, q \in Q$ gilt:

$$p \equiv q \Rightarrow \forall a \in \Sigma : \delta(p, a) \equiv \delta(q, a)$$

⁴Den Zustand q_{11} als Repräsentanten für die dritte Äquivalenzklasse zu wählen ist eine völlig willkürliche Entscheidung. Wir könnten genauso gut q_{20} oder q_{21} wählen.

BEWEIS:

$$\begin{array}{lll}
 p \equiv q & \text{gdw} & \forall w \in \Sigma^* : \tilde{\delta}(p, w) \in F \Leftrightarrow \tilde{\delta}(q, w) \in F \\
 & \text{gdw} & (p \in F \Leftrightarrow q \in F) \wedge \forall a \in \Sigma : \forall w \in \Sigma^* : \tilde{\delta}(p, aw) \in F \Leftrightarrow \tilde{\delta}(q, aw) \in F \\
 \text{impliziert} & & \forall a \in \Sigma : \forall w \in \Sigma^* : \tilde{\delta}(\delta(p, a), w) \in F \Leftrightarrow \tilde{\delta}(\delta(q, a), w) \in F \\
 & \text{gdw} & \forall a \in \Sigma : \delta(p, a) \equiv \delta(q, a) \quad \square
 \end{array}$$

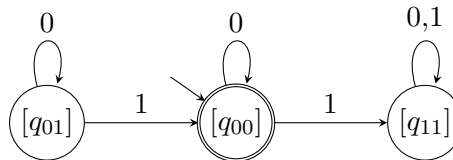
Wir formalisieren das „Verschmelzen“ von Zuständen wie folgt.

Def. 2.7: Der Äquivalenzklassenautomat $\mathcal{A}_{\equiv} = (Q_{\equiv}, \Sigma, \delta_{\equiv}, q^{\text{init}}_{\equiv}, F_{\equiv})$ zu einem DEA $\mathcal{A} = (Q, \Sigma, \delta, q^{\text{init}}, F)$ ist bestimmt durch:

$$\begin{aligned}
 Q_{\equiv} &= \{[q] \mid q \in Q\} \\
 \delta_{\equiv}([q], a) &= [\delta(q, a)] \quad \text{wobei } q \in [q] \text{ beliebig gewählt (also } q_{\equiv} = [q]) \\
 q^{\text{init}}_{\equiv} &= [q^{\text{init}}] \\
 F_{\equiv} &= \{[q] \mid q \in F\} \quad \diamond
 \end{aligned}$$

Vorlesung:
29.04.26

Bsp. 2.6: Der Äquivalenzklassenautomat $\mathcal{A}_{\equiv}$ zum DEA aus Bsp. 2.3 hat das folgende Zustandsdiagramm.



Satz 2.4: Der Äquivalenzklassenautomat ist wohldefiniert und $L(\mathcal{A}_{\equiv}) = L(\mathcal{A})$.

BEWEIS:

1. Wohldefiniert: Es gilt zu zeigen, dass $\delta_{\equiv}([q], a) = [\delta(q, a)]$ nicht abhängig von der Wahl des Repräsentanten $q \in [q]$ ist. Dies folgt direkt aus Lemma 2.3.
2. $L(\mathcal{A}) = L(\mathcal{A}_{\equiv})$: Zunächst zeigen wir per Induktion über w , dass für alle $w \in \Sigma^*$ und alle $q \in Q$ die folgende Äquivalenz gilt.

$$\varphi(w) := \tilde{\delta}(q, w) \in F \Leftrightarrow \tilde{\delta}_{\equiv}([q], w) \in F_{\equiv}$$

I.A. $(\varphi(\varepsilon))$: $\tilde{\delta}(q, \varepsilon) = q \in F \Leftrightarrow \tilde{\delta}_{\equiv}([q], \varepsilon) = [q] \in F_{\equiv}$

I.S. $(\forall a, w : \varphi(w) \Rightarrow \varphi(aw))$:

$$\begin{aligned} \tilde{\delta}(q, aw) \in F &\Leftrightarrow \tilde{\delta}(\delta(q, a), w) \in F \\ &\xLeftrightarrow{I.V.} \tilde{\delta}_{\equiv}([\delta(q, a)], w) \in F_{\equiv} \\ &\Leftrightarrow \tilde{\delta}_{\equiv}(\delta_{\equiv}([q], a), w) \in F_{\equiv} \\ &\Leftrightarrow \tilde{\delta}_{\equiv}([q], aw) \in F_{\equiv} \end{aligned}$$

Damit zeigen wir nun $L(\mathcal{A}) = L(\mathcal{A}_{\equiv})$. Sei $w \in \Sigma^*$.

$$\begin{aligned} w \in L(\mathcal{A}) &\Leftrightarrow \tilde{\delta}(q^{\text{init}}, w) \in F \\ &\Leftrightarrow \tilde{\delta}_{\equiv}([q^{\text{init}}], w) \in F_{\equiv} \\ &\Leftrightarrow w \in L(\mathcal{A}_{\equiv}) \end{aligned}$$

□

In den Übungen werden wir ein Verfahren mit Laufzeit $O(|Q|^4 \cdot |\Sigma|)$ zur Konstruktion des Äquivalenzklassenautomaten kennenlernen. Es gibt aber auch schnellere Verfahren. Mit dem Algorithmus von Hopcroft kann $\mathcal{A}_{\equiv}$ in $O(|Q||\Sigma| \log |Q|)$ erzeugt werden.

Wir werden später ($\rightarrow$ Satz von Myhill-Nerode) sehen, dass $\mathcal{A}_{\equiv}$ ein DEA ist, der $L(\mathcal{A})$ akzeptiert, sodass kein DEA, der ebenfalls $L(\mathcal{A})$ akzeptiert, weniger Zustände haben kann.

Def. 2.8: Eine Äquivalenzrelation $R \subseteq \Sigma^* \times \Sigma^*$ heißt *rechtskongruent*, falls

$$(u, v) \in R \quad \Rightarrow \quad \forall w \in \Sigma^* : (u \cdot w, v \cdot w) \in R. \quad \diamond$$

Motivation: Ist eigentlich jede Sprache regulär? Falls nein, wie können wir nicht-reguläre Sprachen finden? Wie können wir beweisen dass eine Sprache nicht regulär ist? In weiteren Verlauf dieses Unterkapitels wollen wir charakteristische Merkmale von reguläre Sprachen finden. Die Idee dazu basiert auf den folgenden Überlegungen.

Angenommen, wir haben für eine gegebene Sprache L einen ersten Entwurf für einen DEA $\mathcal{A}$ konstruiert. In diesem führen sowohl das Wort $u \in \Sigma^*$ als auch das Wort $v \in \Sigma^*$ in den Zustand q_{42} (formal: $\tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v) = q_{42}$).

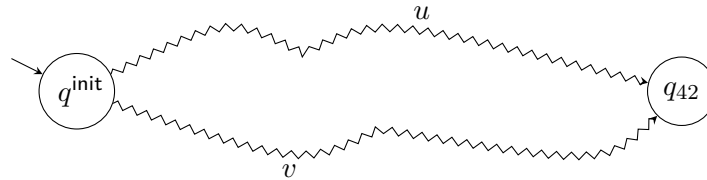


Abb. 2: Schematische Darstellung eines DEA, mit $\tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v) = q_{42}$

- Angenommen, es gibt ein Wort $w \in \Sigma^*$, dass die Äquivalenz $uw \in L \Leftrightarrow vw \in L$ verletzt. Dann muss der Entwurf noch einen Fehler enthalten und wir müssen dafür sorgen, dass q^{init} nicht sowohl von u als auch von v in q_{42} überführt wird.
- Angenommen, es gilt für alle Wörter $w \in \Sigma^*$, dass $uw \in L \Leftrightarrow vw \in L$. Dann wissen wir, dass es eine gute Idee war, q^{init} mit u und v in den gleichen Zustand zu führen.

Angenommen, wir finden nicht nur für zwei Wörter u, v ein Wort w , sodass die Äquivalenz $uw \in L \Leftrightarrow vw \in L$ nicht gilt, sondern wir finden solch ein w für jedes Paar aus einer unendlich großen Menge von Wörtern U .

$$(\text{Formal: } \forall u, v \in U : u \neq v \Rightarrow \exists w^{uv} \in \Sigma^* : \neg(uw^{uv} \in L \Leftrightarrow vw^{uv} \in L))$$

Dann wissen wir, dass all unsere Reparaturversuche hoffnungslos sind, denn der DEA $\mathcal{A}$ bräuchte für jedes $u \in U$ einen eigenen Zustand. Doch ein DEA darf nur endlich viele Zustände haben. Die Sprache L kann also nicht regulär sein.

Im weiteren Verlauf des Unterkapitels werden wir genau diese Überlegungen formal ausarbeiten.

Def. 2.9: Für einen DEA $\mathcal{A}$ definiere

$$R_{\mathcal{A}} = \{(u, v) \mid \tilde{\delta}(q^{\text{init}}, u) = \tilde{\delta}(q^{\text{init}}, v)\}.$$

Beobachtung 1 $R_{\mathcal{A}}$ ist eine Äquivalenzrelation.

Dies folgt daraus, dass „ $=$ “ eine Äquivalenzrelation ist.

Beobachtung 2 $R_{\mathcal{A}}$ ist rechtskongruent.

Dies wird in den Übungen bewiesen.

Beobachtung 3 Wir haben pro Zustand, der von q^{init} erreichbar ist, genau eine Äquivalenzklasse. Der Index von $R_{\mathcal{A}}$ ist also die Anzahl der erreichbaren Zustände.

Für den DEA aus Bsp. 2.3 hat $R_{\mathcal{A}}$ die folgenden Äquivalenzklassen.

$$\begin{aligned} [\varepsilon] &= \{0^n \mid n \in \mathbb{N}\} \\ [1] &= \{0^n 10^m \mid n, m \in \mathbb{N}\} \\ [11] &= \{w \mid \text{Anzahl von Einsen in } w \text{ ist gerade und } \geq 2\} \\ [111] &= \{w \mid \text{Anzahl von Einsen in } w \text{ ist ungerade und } \geq 2\} \end{aligned} \quad \diamond$$

Def. 2.10: Für eine Sprache $L \subseteq \Sigma^*$ ist die *Nerode-Relation* wie folgt definiert.

$$R_L = \{(u, v) \mid \forall w \in \Sigma^* : uw \in L \Leftrightarrow vw \in L\} \quad \diamond$$

Lemma 2.5: Die Nerode-Relation ist eine Äquivalenzrelation.

Dies folgt daraus, dass „ $\Leftrightarrow$ “ eine Äquivalenzrelation ist.

Lemma 2.6: Die Nerode-Relation ist rechtskongruent.

Dafür starten wir mit folgender Beobachtung.

Lemma 2.7: $\forall u, v \in \Sigma^*. \forall a \in \Sigma. (u, v) \in R_L \Rightarrow (ua, va) \in R_L$

BEWEIS:

$$\begin{aligned}
 & (u, v) \in R_L \\
 \Leftrightarrow & \quad \forall w \in \Sigma^* : uw \in L \Leftrightarrow vw \in L \\
 & \quad \text{wähle alle } w \text{ der Form } w = aw' \\
 \Rightarrow & \quad \forall a \in \Sigma : \forall w' \in \Sigma^* : uaw' \in L \Leftrightarrow vaw' \in L \\
 \Leftrightarrow & \quad (ua, va) \in R_L
 \end{aligned}$$

D.h. $(u, v) \in R_L \Rightarrow \forall a \in \Sigma : (ua, va) \in R_L$. □

BEWEIS (Lemma 2.6): Wir wählen die folgende Induktionsbehauptung

$$\forall w \in \Sigma^* : \forall (u, v) \in R_L : (uw, vw) \in R_L$$

und führen eine Induktion über w durch.

I.A. ($w = \varepsilon$): $(u\varepsilon, v\varepsilon) = (u, v) \in R_L$

I.S. ($w \rightsquigarrow a.w$): Zeige nun $\forall (u, v) \in R_L : (uaw, vaw) \in R_L$.

Sei $(u, v) \in R_L$, dann ist nach Lemma 2.7 auch $(ua, va) \in R_L$. Daraus liefert die Induktionsbehauptung sofort $(uaw, vaw) \in R_L$. □

Bsp. 2.7: Sei $\Sigma = \{0, 1\}$. Die Sprache $L = \{w \in \Sigma^* \mid \text{vorletztes Zeichen von } w \text{ ist } 1\}$ hat die folgenden Äquivalenzklassen bezüglich der Nerode-Relation.

$$\begin{aligned}
 [\varepsilon] &= \{w \mid w \text{ endet mit } 00\} \cup \{\varepsilon, 0\} \\
 [1] &= \{w \mid w \text{ endet mit } 01\} \cup \{1\} \\
 [10] &= \{w \mid w \text{ endet mit } 10\} \\
 [11] &= \{w \mid w \text{ endet mit } 11\}
 \end{aligned}$$

Bsp. 2.8: Für ein beliebiges Alphabet Σ gilt:

1. Die Sprache $L = \{\varepsilon\}$ hat genau zwei Äquivalenzklassen bezüglich der Nerode-Relation. Eine Äquivalenzklasse ist $\{\varepsilon\}$, die andere ist Σ^+ .
2. Die Sprache $L = \{\}$ hat genau eine Äquivalenzklasse (nämlich Σ^*) bezüglich der Nerode-Relation.

Bsp. 2.9: Sei $\Sigma = \{0, 1\}$. Die Sprache $L_{\text{centered}} = \{0^n 10^n \mid n \in \mathbb{N}\}$ hat bezüglich der Nerode-Relation die folgende Menge von Äquivalenzklassen:

$$\{[w'] \mid w' \text{ ist Präfix eines Worts } w \in L_{\text{centered}}\} \cup \{[11]\}$$

Bemerkung 1: Die Äquivalenzklasse $[11]$ enthält alle Wörter, die kein Präfix eines Worts aus L_{centered} sind.

Bemerkung 2: Nicht für alle Präfixe von Wörtern in L_{centered} sind die zugehörigen Äquivalenzklassen paarweise verschieden. Es gilt z.B. $[01] = [0010]$.

Bemerkung 3: Für je zwei verschiedene $k \in \mathbb{N}$ sind die Äquivalenzklassen $[0^k 1]$ verschieden. Es gibt also unendlich viele Äquivalenzklassen.

Bsp. 2.10: Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) > \#_b(w)\}$ hat bezüglich der Nerode-Relation die folgende Menge von Äquivalenzklassen:

$$\{[w] \mid \#_a(w) - \#_b(w) = k, k \in \mathbb{Z}\}$$

Satz 2.8 (Myhill und Nerode): Für Sprachen $L \subseteq \Sigma^*$ sind folgende Aussagen äquivalent.

1. L wird von einem DEA akzeptiert.
2. L ist Vereinigung von Äquivalenzklassen einer rechtskongruenten Äquivalenzrelation mit *endlichem* Index.
3. Die Nerode-Relation R_L hat *endlichen* Index.

BEWEIS: Wir beweisen die paarweise Äquivalenz in drei Schritten:

$$(1) \Rightarrow (2), \quad (2) \Rightarrow (3) \quad \text{und} \quad (3) \Rightarrow (1)$$

(1) $\Rightarrow$ (2) Sei $\mathcal{A}$ ein DEA. Es gilt

$$L(\mathcal{A}) = \{w \mid \tilde{\delta}(q^{\text{init}}, w) \in F\} = \bigcup_{q \in F} \{w \mid \tilde{\delta}(q^{\text{init}}, w) = q\}.$$

Nun sind die Mengen $\{w \mid \tilde{\delta}(q^{\text{init}}, w) = q\}$ für erreichbare $q \in Q$ genau die Äquivalenzklassen der Relation $R_{\mathcal{A}}$ aus Def. 2.9, einer rechtskongruenten Äquivalenzrelation. Der Index von $R_{\mathcal{A}}$ ist die Anzahl der *erreichbaren* Zustände und somit endlich: $\text{Index}(R_{\mathcal{A}}) \leq |Q| < \infty$.

(2) $\Rightarrow$ (3) Sei R eine rechtskongruente Äquivalenzrelation mit endlichem Index, sodass L die Vereinigung von R -Äquivalenzklassen ist.

Es genügt zu zeigen, dass die Nerode-Relation R_L eine Obermenge von R ist.⁵

$$\begin{aligned} (u, v) \in R &\Rightarrow u \in L \Leftrightarrow v \in L, \quad \text{da } L \text{ Vereinigung von Äquivalenzklassen ist} \\ &\Rightarrow \forall w \in \Sigma^* : uw \in L \Leftrightarrow vw \in L, \quad \text{da } R \text{ rechtskongruent} \\ &\Rightarrow (u, v) \in R_L, \quad \text{nach Definition der Nerode-Relation} \end{aligned}$$

Es gilt also $R \subseteq R_L$ und somit $\text{Index}(R_L) \leq \text{Index}(R) < \infty$.

(3) $\Rightarrow$ (1) Gegeben R_L , konstruiere $\mathcal{A} = (Q, \Sigma, \delta, q^{\text{init}}, F)$

- $Q = \{[w]_{R_L} \mid w \in \Sigma^*\}$ endlich, da $\text{Index}(R_L)$ endlich
- $\delta(q, a) = [wa]$ für ein $w \in q$ —wohldefiniert (vgl. Lemma 2.7)
- $q^{\text{init}} = [\varepsilon]$
- $F = \{[w] \mid w \in L\}$

Wir wollen nun $L(\mathcal{A}) = L$ zeigen. Dafür beweisen wir zunächst per Induktion über w die folgende Eigenschaft.

$$\forall w \in \Sigma^* : \forall v \in \Sigma^* : \tilde{\delta}([v], w) = [v \cdot w]$$

I.A. ($w = \varepsilon$): $\tilde{\delta}([v], \varepsilon) = [v] = [v \cdot \varepsilon]$

I.S. ($w \rightsquigarrow a.w$)

$$\begin{aligned} \tilde{\delta}([v], a.w) &= \tilde{\delta}(\delta([v], a), w) \\ &= \tilde{\delta}([v \cdot a], w) \\ &\stackrel{\text{I.V.}}{=} [(v \cdot a) \cdot w] \\ &= [v \cdot a.w] \end{aligned}$$

⁵Zur Erklärung: Falls $R \subseteq R_L$, dann gilt $\text{Index}(R) \geq \text{Index}(R_L)$. Intuitiv: Je mehr Elemente eine Äquivalenzrelation R enthält, desto mehr Elemente sind bzgl. dieser Relation äquivalent, d.h. desto weniger unterschiedliche Äquivalenzklassen gibt es.

Nun zeigen wir $L(\mathcal{A}) = L$ wie folgt:

$$\begin{aligned} w \in L(\mathcal{A}) &\text{ gdw } \tilde{\delta}([\varepsilon], w) \in F \\ &\text{ gdw } [w] \in F, \quad (\text{per Induktion gezeigte Eigenschaft für } v = \varepsilon) \\ &\text{ gdw } w \in L \end{aligned} \quad \square$$

Korollar 2.5: Der im Beweisschritt (3) $\Rightarrow$ (1) konstruierte Automat $\mathcal{A}$ ist ein minimaler Automat (bzgl. der Zustandsanzahl) für eine reguläre Sprache L . $\diamond$

BEWEIS: Sei $\mathcal{A}' = (Q', \dots)$ ein *beliebiger* DEA mit $L(\mathcal{A}') = L$.

Aus „1 $\Rightarrow$ 2“ wissen wir, dass $\text{Index}(R_{\mathcal{A}'}) \leq |Q'|$ gilt.

Aus „2 $\Rightarrow$ 3“ wissen wir, dass $R_{\mathcal{A}'} \subseteq R_L$ und somit $\text{Index}(R_L) \leq \text{Index}(R_{\mathcal{A}'})$ gilt.

Aus „3 $\Rightarrow$ 1“ erhalten wir $\mathcal{A} = (Q, \dots)$ mit $|Q| = \text{Index}(R_L)$.

Wegen $|Q| = \text{Index}(R_L) \leq \text{Index}(R_{\mathcal{A}'}) \leq |Q'|$ hat $\mathcal{A}$ nicht mehr Zustände als $\mathcal{A}'$. $\square$

2.3 Nichtdeterministischer endlicher Automat (NEA)

Aufgabe: Konstruiere für eine natürliche Zahl $n \in \mathbb{N}$ einen DEA für die folgende Sprache.

Vorlesung:
12.05.2026

$$L_n = \{w \in \{0, 1\}^* \mid |w| \geq n \text{ und das } n\text{-letzte Symbol von } w \text{ ist } 1\}$$

Naiver Lösungsversuch:

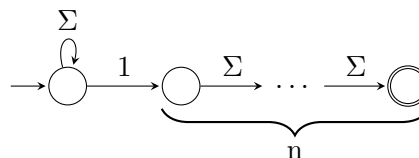


Abb. 3: Nichtdeterministischer Automat für L_n

Problem: Das Zustandsdiagramm beschreibt keinen DEA: Der Startzustand hat zwei ausgehende Kanten für 1.

Untersuche die Sprache mit Hilfe der Nerode-Relation. Beobachtung: Je zwei Wörter der Länge n sind in unterschiedlichen Äquivalenzklassen. Es gibt also mindestens 2^n Äquivalenzklassen; aus Korollar 2.5 wissen wir, dass ein minimaler DEA, der L_n akzeptiert, mindestens 2^n Zustände haben muss.

Idee: Definiere eine neue Art von Automaten, bei dem ein Zustand pro Zeichen mehrere Nachfolger haben darf.

Def. 2.11 (NEA): Ein *nichtdeterministischer endlicher Automat* (NEA)⁶ ist ein 5-Tupel

$$\mathcal{N} = (\Sigma, Q, \delta, q^{\text{init}}, F).$$

Dabei ist

- Σ ein Alphabet,
- Q eine *endliche* Menge, deren Elemente wir *Zustände* nennen,
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ eine Funktion, die wir *Transitionsfunktion* nennen,
- $q^{\text{init}} \in Q$ ein Zustand, den wir *Startzustand* nennen und
- $F \subseteq Q$ eine Teilmenge der Zustände, deren Elemente wir *akzeptierende Zustände* nennen. $\diamond$

Bemerkung: Die Definition des NEA unterscheidet sich vom DEA nur in der Transitionsfunktion. Beim DEA ist der Bildbereich der Transitionsfunktion die Menge der Zustände Q . Beim NEA ist der Bildbereich die Potenzmenge $\mathcal{P}(Q)$ der Zustandsmenge Q . Analog zu DEAs werden wir auch NEAs mit Hilfe eines Zustandsdiagramms beschrieben. So beschreibt Abb. 3 für jedes $n \in \mathbb{N}$ einen NEA für die Sprache L_n .

Im Folgenden sei $\mathcal{N}$ immer ein NEA.

Def. 2.12 (Lauf eines Automaten): Die Funktion $\text{run} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q^*)$ ist so definiert, dass $\text{run}(q, w)$ die Folgen von Zuständen liefert, die $\mathcal{N}$ vom Zustand q angesetzt auf die Eingabe w durchläuft.

$$\text{run}(q, \varepsilon) = \{q\} \tag{1}$$

$$\text{run}(q, a.w) = \{q.\bar{q} \mid \bar{q} \in \text{run}(q', w), q' \in \delta(q, a)\} \tag{2}$$

Ein Lauf $q_0 \dots q_n \in \text{run}(q_0, w)$ heißt *initial*, falls $q_0 = q^{\text{init}}$. Er heißt *akzeptierend*, falls $q_n \in F$. $\diamond$

Def. 2.13 (Akzeptanz): Ein Wort $w \in \Sigma^*$ wird von $\mathcal{N}$ *akzeptiert*, falls $\mathcal{N}$ einen initialen und akzeptierenden Lauf über w hat. Die von $\mathcal{N}$ akzeptierte Sprache ist die Menge der von $\mathcal{N}$ akzeptierten Wörter, d.h.

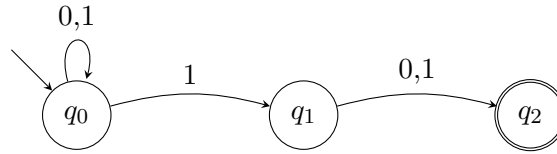
$$L(\mathcal{N}) = \{w \in \Sigma^* \mid \exists \text{ initialer, akzeptierender Lauf von } \mathcal{N} \text{ über } w\}. \quad \diamond$$

Bsp. 2.11: Der NEA für die Sprache

$$L_2 = \{w \in \{0, 1\}^* \mid \text{das zweitletzte Zeichen von } w \text{ ist } 1\}$$

hat die folgende graphische Repräsentation.

⁶engl. NFA $\triangleq$ nondeterministic finite automaton



Bemerkung: Die Frage, ob ein gegebenes Wort w akzeptiert wird (das „Wortproblem“), lässt sich für NEAs nicht mehr so leicht beantworten wie wir es von DEAs gewohnt sind. Ein sinnvolles Vorgehen scheint, jeden initialen Lauf zu betrachten, doch z.B. für das Wort 11 hat obiger NEA bereits drei verschiedene initiale Läufe: $q_0q_0q_0$, $q_0q_0q_1$ und $q_0q_1q_2$.

Bemerkung: Die Definitionen von NEA und DEA in der Literatur sind nicht einheitlich. Es gibt äquivalente NEA-Definitionen, die statt der Transitionsfunktion $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ eine Transitionsrelation $\delta \subseteq Q \times \Sigma \times Q$ verwenden. Es gibt alternative NEA-Definitionen, die eine Menge von Startzuständen erlauben. Alternativ könnte man auch zunächst den NEA einführen und den DEA als Spezialfall dessen definieren (Spezialfall: Das Bild der Transitionsfunktion ist einelementig für alle $q \in Q$ und $a \in \Sigma$).

Bemerkung: Zu jedem DEA $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ gibt es einen NEA, der die gleiche Sprache akzeptiert. Beispiel: der NEA $\mathcal{N} = (\Sigma, Q, \delta_{\text{NEA}}, q^{\text{init}}, F)$ mit $\delta_{\text{NEA}}(q, a) = \{\delta(q, a)\}$, der sich von $\mathcal{A}$ nur in der Transitionsfunktion unterscheidet.

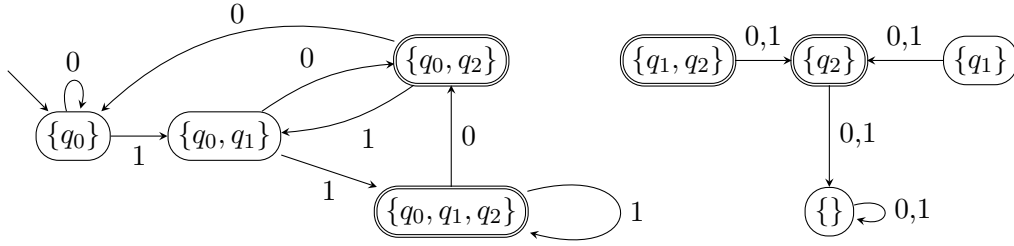
Satz 2.6 (Rabin und Scott): Zu jedem NEA $\mathcal{N}$ mit n Zuständen gibt es einen DEA $\mathcal{A}_{\mathcal{P}}$ mit 2^n Zuständen, sodass $L(\mathcal{A}_{\mathcal{P}}) = L(\mathcal{N})$ gilt.

Zur Vorbereitung des Beweises machen wir zunächst die folgende Definition.

Def. 2.14 (Potenzmengenautomat): Für einen gegebenen NEA $\mathcal{N} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ist der Potenzmengenautomat $\mathcal{A}_{\mathcal{P}}$ wie folgt definiert.

$$\begin{aligned}
 Q_{\mathcal{P}} &= \mathcal{P}(Q) \\
 \delta_{\mathcal{P}}(p, a) &= \bigcup_{q \in p} \{\delta(q, a)\} \\
 q_{\mathcal{P}}^{\text{init}} &= \{q^{\text{init}}\} \\
 F_{\mathcal{P}} &= \{p \in Q_{\mathcal{P}} \mid p \cap F \neq \emptyset\}
 \end{aligned}
 \quad \diamond$$

Bsp. 2.12: Der Potenzmengenautomat für den NEA aus Bsp. 2.11 hat das folgende Zustandsdiagramm.



Die vier Zustände auf der rechten Seite sind nicht erreichbar.

BEWEIS (von Satz 2.6): Zeige $L(\mathcal{A}_{\mathcal{P}}) = L(\mathcal{N})$. Dafür beweisen wir zunächst per Induktion über w die folgende Eigenschaft:

$$\forall w \in \Sigma^* : \forall p \in Q_{\mathcal{P}} \setminus \{\{\}\} : \forall q \in Q :$$

$$q \in \tilde{\delta}_{\mathcal{P}}(p, w) \Leftrightarrow \exists q_0 \in p. q_0, q_1, \dots, q_n \in \text{run}(q_0, w), \text{ sodass } q_n = q$$

I.A. ($w = \varepsilon$): $q \in \tilde{\delta}_{\mathcal{P}}(p, \varepsilon) = p$ wähle $q_0 = q$.

I.S. ($w = aw'$) ein beliebiges Wort.

$$\begin{aligned} q \in \tilde{\delta}_{\mathcal{P}}(p, w) &\Leftrightarrow q \in \tilde{\delta}_{\mathcal{P}}(\delta_{\mathcal{P}}(p, a), w') \\ &\stackrel{\text{I.V.}}{\Leftrightarrow} \exists q_1 \in \delta_{\mathcal{P}}(p, a). q_1, q_2, \dots, q_n \in \text{run}(q_1, w'), \text{ sodass } q_n = q \\ &\Leftrightarrow \exists q_0 \in p. q_1 \in \delta_{\mathcal{P}}(p, a). q_1, q_2, \dots, q_n \in \text{run}(q_1, w'), \text{ sodass } q_n = q \\ &\Leftrightarrow \exists q_0 \in p. q_0, q_1, q_2, \dots, q_n \in \text{run}(q_1, aw'), \text{ sodass } q_{n+1} = q \end{aligned}$$

Mit Hilfe dieser Eigenschaft zeigen wir nun die Gleichheit $L(\mathcal{A}_{\mathcal{P}}) = L(\mathcal{N})$.

$$\begin{aligned} w \in L(\mathcal{A}_{\mathcal{P}}) &\Leftrightarrow \tilde{\delta}_{\mathcal{P}}(q_{\mathcal{P}}^{\text{init}}, w) \in F_{\mathcal{P}} \\ &\Leftrightarrow \exists p_f \in F_{\mathcal{P}} : \tilde{\delta}_{\mathcal{P}}(q_{\mathcal{P}}^{\text{init}}, w) = p_f \\ &\Leftrightarrow \exists q_f \in F : q_f \in \tilde{\delta}_{\mathcal{P}}(q_{\mathcal{P}}^{\text{init}}, w) \\ &\Leftrightarrow \exists q_0 \in q_{\mathcal{P}}^{\text{init}} = \{q_{\mathcal{A}}^{\text{init}}\}. q_0, q_1, \dots, q_n \in \text{run}(q_0, w), \text{ sodass } q_n = q_f \in F \\ &\Leftrightarrow \exists \text{ initialer, akzeptierender Lauf von } \mathcal{N} \text{ über } w \\ &\Leftrightarrow w \in L(\mathcal{N}) \end{aligned}$$

□

Bemerkung: Es gelten also die folgenden Äquivalenzen.

$$L \text{ regulär} \stackrel{\text{Def. 2.3}}{\Leftrightarrow} L = L(\mathcal{A}) \text{ für einen DEA } \mathcal{A} \iff L = L(\mathcal{N}) \text{ für einen NEA } \mathcal{N}$$

Bemerkung: NEAs sind eine exponentiell kompaktere Repräsentation von regulären Sprachen im folgenden Sinne:

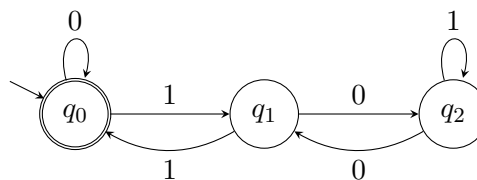
1. Es gibt mit L_n (n -letztes Zeichen) eine Menge von Sprachen, die sich durch einen NEA mit $n + 1$ Zuständen darstellen lassen, aber bei denen ein minimaler DEA mindestens 2^n Zustände hat. (Siehe Übungsblatt 3, Aufgabe 2).
2. Zu jedem NEA mit n Zuständen gibt es einen DEA mit 2^n Zuständen, der die gleiche Sprache akzeptiert. (Satz 2.6).
3. Zu jedem DEA mit n Zuständen gibt es einen NEA mit n Zuständen, der die gleiche Sprache akzeptiert.

2.4 Pumping Lemma (PL) für reguläre Sprachen

Notation: Sei $\text{bin} : \{0, 1\}^* \rightarrow \mathbb{N}$ die Decodierung von Bitstrings in natürliche Zahlen; z.B. $\text{bin}(101) = 5$, $\text{bin}(\varepsilon) = 0$.

Vorlesung:
13.05.2026

Bsp. 2.13: Betrachte den folgenden DEA, der die Sprache der Binärcodierung von durch drei teilbaren Zahlen akzeptiert: $L = \{w \in \{0, 1\}^* \mid \text{bin}(w) \equiv_3 0\}$



Beobachtungen:

- Es gilt offensichtlich, dass $11 \in L$.
- Es gilt auch, dass $1001 \in L$.
- Der Automat hat eine Schleife bei $\delta(q_1, 00) = q_1$, die mehrfach „abgelaufen“ werden kann, ohne die Akzeptanz zu beeinflussen.
- Also gilt auch $100001 \in L$.
- Im Allgemeinen gilt $\forall i \in \mathbb{N} : 1(00)^i 1 \in L$.

Verdacht: Alle „langen“ Wörter lassen sich in der Mitte „aufpumpen“. Wir formalisieren diesen Verdacht im folgenden Lemma.

Lemma 2.7 (Pumping Lemma): Sei L eine reguläre Sprache. Dann gilt:

$$\begin{aligned}
 &\exists n \in \mathbb{N}, n > 0 : \quad \forall z \in L, |z| \geq n : \\
 &\quad \exists u, v, w \in \Sigma^* : \\
 &\quad z = uvw, |uv| \leq n, |v| \geq 1 \\
 &\text{und } \forall i \in \mathbb{N} : uv^i w \in L
 \end{aligned}$$

BEWEIS: Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\text{init}}, F)$ ein beliebiger DEA für L . Wähle $n = |Q|$ und $z \in L$ beliebig mit $|z| \geq n$.

Beim Lesen von z durchläuft $\mathcal{A}$ genau $\overbrace{|z| + 1}^{\geq n+1}$ Zustände. Somit gibt es mindestens einen Zustand $q \in Q$, der mehrmals besucht wird (Schubfachprinzip).

Wähle den Zustand q , der als Erster zweimal besucht wird.

$$\begin{aligned} \text{Nun gilt: } \quad \exists u : \tilde{\delta}(q^{\text{init}}, u) = q & \quad u \text{ Präfix von } z \\ \exists v : \quad \tilde{\delta}(q, v) = q & \quad uv \text{ Präfix von } z \\ \exists w : \quad \tilde{\delta}(q, w) \in F & \quad uvw = z \\ & |v| \geq 1 \\ & |uv| \leq n \quad \text{da } q \text{ als Erster zweimal besucht wird} \end{aligned}$$

$$\begin{aligned} \text{Es folgt für beliebiges } i \in \mathbb{N} : \quad \tilde{\delta}(q^{\text{init}}, uv^i w) &= \tilde{\delta}(q, v^i w) \\ &= \tilde{\delta}(q, w) \quad \text{denn } \forall i : \tilde{\delta}(q, v^i) = q \\ &\in F \quad \square \end{aligned}$$

Bsp. 2.14: Die Sprache $L_{\text{centered}} = \{0^n 10^n \mid n \in \mathbb{N}\}$ ist nicht regulär.

Wir geben hierfür einen Widerspruchsbeweis mit Hilfe des Pumping Lemmas PL.

Sei n die Konstante aus dem PL. Wähle $z = 0^n 10^n$. (Gültige Wahl, da $|z| = 2n + 1 \geq n$)
Laut PL existieren u, v, w , sodass $z = uvw$ mit $|v| \geq 1, |uv| \leq n$ und $\forall i \in \mathbb{N} : uv^i w \in L$.
Nach Wahl von z gilt nun

- $uv = 0^m$ mit $m \leq n$
- $v = 0^k$ mit $k \geq 1$
- $w = 0^{n-m} 10^n$

Betrachte $uv^2w = 0^{m-k} 0^{2k} 0^{n-m} 10^n = 0^{n+k} 10^n \notin L$. Widerspruch! Somit ist L nicht regulär.

Zur Illustration:

$$\begin{array}{c} \underbrace{0 \dots 0}_n \underbrace{10 \dots 0}_n \\ \text{---u---v---w---} \end{array}$$

2.5 ε -Transitionen

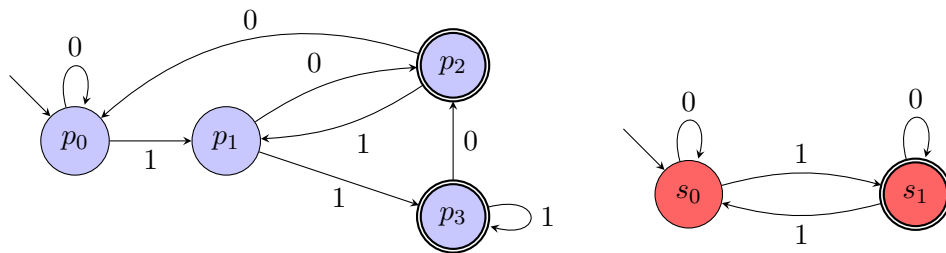
In diesem Unterkapitel führen wir mit dem ε -NEA ein weiteres Automatenmodell ein. Wir wollen ε -NEAs zunächst durch die folgende Fragestellung und anschließende Diskussion motivieren.

Frage: Gegeben zwei reguläre Sprachen L_1, L_2 , ist auch die Konkatination $L_1 \cdot L_2$ eine reguläre Sprache?

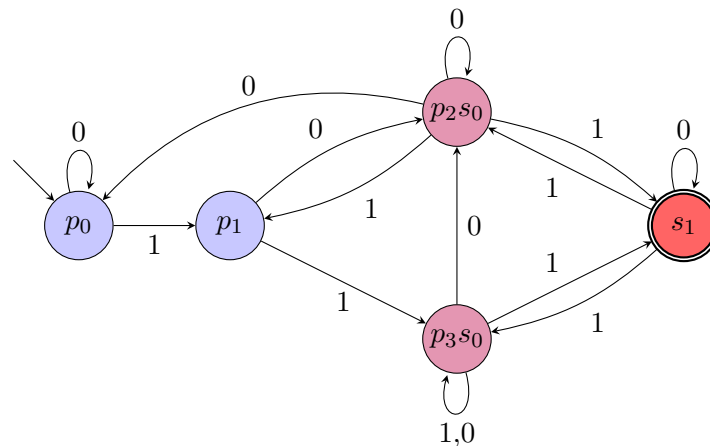
Idee: Gegeben DEA $\mathcal{A}_1$ mit $L(\mathcal{A}_1) = L_1$ und DEA $\mathcal{A}_2$ mit $L(\mathcal{A}_2) = L_2$, konstruiere NEA für $L_1 \cdot L_2$ durch „Hintereinanderschalten“ von $\mathcal{A}_1$ und $\mathcal{A}_2$; immer wenn wir in einem akzeptierenden Zustand von $\mathcal{A}_1$ sind, erlauben wir, in $\mathcal{A}_2$ zu „wechseln“.

Erste, naive (und inkorrekte) Umsetzung dieser Idee: Verschmelze akzeptierende Zustände von $\mathcal{A}_1$ mit dem Startzustand von $\mathcal{A}_2$. Wir betrachten die folgenden Automaten, um zu sehen, dass diese Umsetzung nicht zielführend ist.

Bsp. 2.15: Links: DEA $\mathcal{A}_1$, der Automat aus Bsp. 2.11 eingeschränkt auf die erreichbaren Zustände. Rechts: DEA $\mathcal{A}_2$ mit der Sprache $\{w \in \{0, 1\}^* \mid \text{Anzahl } 1 \text{ in } w \text{ ungerade}\}$.



Unten: NEA $\mathcal{N}_{\text{naiv}}$ aus der naiven und inkorrekten Konstruktion für die Konkatination.



Dieser NEA akzeptiert nun auch das Wort $w = 11011$. Allerdings ist w nicht in der Konkatenation $L(\mathcal{A}_1) \cdot L(\mathcal{A}_2)$, denn es gibt keine Zerlegung $w = w_1 \cdot w_2$, sodass sowohl das Präfix w_1 von $\mathcal{A}_1$ als auch das Suffix w_2 von $\mathcal{A}_2$ akzeptiert wird.

Das „Verschmelzen“ von p_2 (bzw. p_3) mit s_0 war also keine gute Idee. Was uns aber helfen würde: ein Zustandsübergang, der es uns erlaubt, von Zustand p_2 (bzw. p_3) in den Zustand s_0 zu gehen, ohne dabei ein Zeichen zu lesen.

Wir nennen solch einen Zustandsübergang ε -Transition und definieren einen Automaten, der solche Zustandsübergänge haben kann, wie folgt.